

Picorules: Composing Computable Clinical Phenotypes

Reusable clinical constructs from the warehouse to the consultation

Asanga Sanjaya Abeyaratne

Creator and Lead Developer, Picorules
asaabey@gmail.com

September 2026

Abstract

A clinical description of a patient may bring together current measurements, their history, persistent abnormalities, and related conditions. Each characteristic needs a definition, and the same definition may contribute to several descriptions or applications. Picorules provides a small language for expressing reusable clinical constructs and composing them into computable phenotype definitions.

The language retrieves dated patient facts, summarises their history, and applies classification rules. Named outputs can be reused by other ruleblocks, allowing authors to build richer definitions from smaller constructs. A shared parser supports SQL generation for Oracle, SQL Server, and PostgreSQL, together with direct evaluation in JavaScript. Data adapters supply the JavaScript evaluator with records from EADV, FHIR, or openEHR sources. These execution paths let a shared definition serve population analysis and individual-patient applications.

This paper explains the design through kidney disease examples and describes its origins in Territory Kidney Care. Published studies provide evidence for the deployed clinical algorithms; a separate test harness exercises the newer adapters and runtime. Agreement on selected outputs does not establish that every composition is clinically meaningful or that every execution path is equivalent. The paper sets out the mechanisms available for composing definitions, the assumptions on which reuse depends, and the evaluation still needed.

Contents

1	From observations to clinical phenotypes	3
2	What existing tools provide	4
3	From a clinical construct to its execution	6
4	Expressing and composing clinical constructs	7
5	Executing the same source in SQL and JavaScript	10
6	Connecting the rules to patient data	12
7	Comparing the work an author has to do	14
8	Using AI to help write rules	15
9	Territory Kidney Care: where the design began	16
10	Using phenotype definitions in clinical applications	17
11	Where the design needs care	18
12	The next useful steps	19
13	Conclusion	20

1 From observations to clinical phenotypes

Consider a kidney service describing a patient’s renal health. The latest renal function measurement is one part of that description. Change over time, persistence of abnormal findings, albuminuria, and related conditions may also matter. Each characteristic needs a definition. Several can contribute to other descriptions of the same patient, so it is useful to express them as concepts that can be combined and reused.

In this paper, a *clinical construct* is a named interpretation or derivation from patient evidence, such as a measurement summary or an assessment of persistence. A *computable phenotype definition* specifies how observed and derived characteristics are combined into a meaningful description of a patient. Evaluating that definition produces a result for a patient at a specified time. That result may contain several characteristics rather than a single disease flag.

Role	Example	Purpose
Observation	A dated eGFR measurement	Supply recorded evidence
Clinical construct	A summary of renal function over time	Interpret selected evidence
Phenotype definition	A renal profile combining function, albuminuria, and related conditions	Compose a clinical description
Application	A registry view or consultation display	Use the description for a particular task

These roles are not additional Picorules data types. They explain the intended use of its rules and named outputs. A construct can contribute to several phenotypes, and a phenotype can supply characteristics to a broader definition. Applications may add decision criteria of their own: a phenotype can inform treatment eligibility without itself specifying a treatment decision.

The population registry and consultation application may need the same definitions. Both depend on the same interpretation of the patient’s evidence, but they often arrive at separate pieces of software.

The registry may use SQL written by an analyst. The consultation application may use JavaScript written by a developer. Each implementation must decide which observations count, how to order them, and what to do when a result is missing. A small change to the definition can require two sets of edits and two rounds of checking. Even when the implementations agree today, someone has to keep them in agreement tomorrow.

Picorules grew out of maintaining clinical calculations in an Oracle database for Territory Kidney Care (TKC). In 2019, deployment restrictions made the database the available place to run the software. I built the first compiler in PL/SQL so that it could work within that constraint. It read a short description of a calculation, generated SQL, and ran it inside Oracle. That origin explains much of the language: it works with dated observations, builds calculations from small steps, and allows one ruleblock to use another’s results.

The TypeScript implementation extends that design. It generates SQL for several databases and can evaluate the parsed rules in JavaScript. This makes it possible to reuse clinical definitions in an application as well as in a warehouse. The broader aim is to compose meaningful patient

descriptions from reusable constructs. Portability supports that aim by allowing those definitions to be evaluated in different settings.

1.1 Getting the right data

Clinical information comes in forms suited to the systems that collect it. A warehouse might store observations as rows containing a patient identifier, attribute, date, and value. FHIR stores them in typed resources, often with nested fields and references. An openEHR repository organises them through archetypes and templates. Older systems have their own arrangements.

These differences survive the adoption of a common exchange standard. The dates, codes, units, and status fields still need interpretation. Work on EAV databases and transformations between i2b2 and OMOP illustrates the effort involved in bringing clinical data into a usable form [Dinu et al., 2007, Klann et al., 2019].

For a rule author, the useful question is usually quite specific: what is the latest eGFR, or how many qualifying observations occurred during the past year? Picorules gives these questions a stable expression. An adapter or a database mapping must then supply the records with the meaning the rule expects.

1.2 Keeping clinical meaning open to review

A short rule is useful only if its meaning is clear. A clinician reviewing a phenotype definition should be able to find its constituent constructs, the observations they use, and their assumptions about time and missing data. Constructs that work separately may rely on incompatible assumptions when combined. Inspecting those assumptions still requires familiarity with the language. Readability is a design aim, and needs to be tested with its intended users.

The surrounding work also matters. Reviews of CDS implementation describe difficulties with workflow, trust, and organisational support [Abell et al., 2023, Liberati et al., 2017]. A language can make a clinical definition easier to inspect; it cannot, on its own, decide whether an alert belongs in a consultation or whether a service has the staff to act on it.

The following sections explain how the language expresses and connects clinical constructs, then examine its execution paths, adapter tests, and TKC deployment. The aim is to make this approach to composing phenotypes concrete enough to assess, including the places where it still needs evidence.

2 What existing tools provide

Clinical decision support involves several jobs. Someone must express the clinical logic, connect it to patient data, and decide when to present the result. Some tools cover one of these jobs; others combine them. Comparing the tools is easier when we keep those responsibilities in view.

2.1 CQL: a language for clinical expressions

Clinical Quality Language (CQL) provides a standard way to describe clinical expressions, with a type system, reusable libraries, temporal operations, and terminology support. Its specification allows different data models, including QDM and FHIR [Health Level Seven International, 2025]. This is an important point of comparison: data model independence is already a stated goal of CQL.

A CQL library can nevertheless depend on a particular model. Expressions written over FHIR resources refer to FHIR types and properties. Moving those expressions to another model requires suitable mappings or changes to the expressions. Picorules places that dependency behind its attribute vocabulary: the rule asks for a named series of dated values, and the adapter resolves it. This reduces the data structure visible to the author, while giving the adapter more responsibility.

There is published experience of CQL phenotyping across platforms [Brandt et al., 2020], as well as work on repositories of reusable definitions [Brandt et al., 2021]. These are relevant precedents. Picorules should be assessed alongside them on the effort required to author, map, test, and maintain equivalent phenotype definitions and their constituent constructs.

The practical focus here is composing clinical definitions through named outputs and bindings, supported by SQL generation for an EADV warehouse and JavaScript evaluation of the same source. This is a description of the Picorules implementation. It does not establish that population evaluation or other compilation strategies are unavailable to CQL.

2.2 CDS Hooks and SMART on FHIR: delivering the result

CDS Hooks defines exchanges between an EHR and a decision support service at specified points in a workflow. A service receives context and may return cards containing information or suggested actions. The service still needs an implementation of its clinical logic. Picorules can evaluate the clinical constructs and phenotype definitions used by the service.

SMART on FHIR supports applications that launch with an EHR context and obtain authorised access to patient data. Such an application can embed the Picorules JavaScript evaluator after fetching the records it needs. Authentication, access control, data retrieval, and the presentation of results remain application responsibilities.

FHIR’s Clinical Reasoning resources address a further part of the problem: representing and exchanging computable knowledge artefacts. Using a small embedded evaluator does not remove the need for that surrounding infrastructure when a deployment requires it. Surveys of CDS standards describe combinations of these approaches in practice [Taber et al., 2021].

2.3 Arden Syntax: experience with portable rules

Arden Syntax organises clinical logic into Medical Logic Modules, with provisions for data access, logic, and actions. Its experience is relevant to any claim about sharing clinical rules: moving the logic is easier than ensuring that a new site supplies equivalent data.

The familiar “curly braces problem” concerns the institution-specific data retrieval placed inside Arden modules. Picorules gives data access a defined interface and a common attribute vocabulary. This makes the boundary explicit, but a new site still has to map its records to that vocabulary and check the result. Comparisons of Arden and other rule formalisms show why the representation and data binding both deserve attention [Soares et al., 2021, Iglesias et al., 2020].

2.4 Decision tables, rule engines, and guideline models

Other approaches offer different starting points. DMN represents decisions in tables, while BPMN describes processes. Rule engines such as Drools support inference over supplied facts. Guideline formalisms including GLIF, PROforma, and Asbru represent plans and their relationships; openEHR’s Guideline Definition Language connects rules with archetype-based clinical

models. The history of these approaches contains useful lessons about expressiveness, knowledge acquisition, and deployment [Peleg, 2013, Papadopoulos et al., 2022].

Picorules builds clinical constructs by retrieving dated facts, deriving values, and sharing those values between ruleblocks. These operations support the composition of phenotype definitions from characteristics such as longitudinal laboratory summaries and threshold classifications. Workflow and general inference remain responsibilities of other software. Whether these constructs are sufficient for a particular phenotype or guideline has to be established by working through its requirements.

3 From a clinical construct to its execution

A phenotype definition depends on smaller constructs whose meaning must remain clear when they are reused. Start with one of the simplest: the latest recorded eGFR.

```
egfr => eadv.lab_bld_egfr.val.last();
```

Read from left to right, this names a result, identifies the eGFR observations, selects their values, and asks for the latest one. It says nothing about a table join or a FHIR resource path. Those choices become necessary when the rule is run.

In the SQL path, the compiler turns the request into a query that selects the latest matching observation for each patient. In the JavaScript path, the evaluator asks an adapter for the matching records and applies the aggregation in memory. Both start from the parsed form of the same line.

Three parts of the system support this arrangement:

Part	Responsibility	Example
Language	Define and compose clinical constructs	Retrieve the latest eGFR and reuse a named result
Execution backend	Evaluate the definitions	Generate SQL or interpret the rule in JavaScript
Data mapping	Supply facts with the expected meaning	Match a warehouse attribute or a FHIR terminology code

The SQL backend assumes an EADV-shaped database. The pluggable `DataAdapter` interface belongs to the JavaScript evaluator. An adapter therefore does not automatically make generated SQL work against an arbitrary database schema. A warehouse with another schema needs an appropriate mapping or preparation step.

3.1 A small interface with a substantial responsibility

The JavaScript evaluator uses this contract:

```
interface DataAdapter {
  getRecords(attributeList: string[]): DataRecord[];
}
```

```
interface DataRecord {  
  val: number | string | Date | null;  
  dt: Date | null;  
}
```

An adapter returns the dated values for one patient's requested attributes. The evaluator does not need to know how those records were stored. For a source reached over a network, data must be fetched before this synchronous method can return it.

The interface is deliberately small. Implementing it correctly can still take considerable work. The adapter must know which source fields represent the clinical attribute, which date to use, and which records should be included. Units and terminology must agree with the rule's assumptions. Returning a number and a date is easy; returning the intended measurement is the part that needs care.

The current implementations cover in-memory EADV records, FHIR R4 Bundles, and openEHR query results. The next sections explain the language and backends before returning to those adapters in more detail.

4 Expressing and composing clinical constructs

A ruleblock is a named unit of executable definitions. It starts by retrieving the facts it needs, then derives named results from them. A block may express one clinical construct or several related characteristics; a larger phenotype definition may draw on more than one block. Documentation and output labels sit alongside the logic so that a reader can follow both the definition and its purpose.

The recurring tasks are fairly small: find an observation, summarise a history, apply a threshold, or use a result from another ruleblock. These operations supply the components of richer clinical descriptions. Keeping them explicit helps a reviewer follow how a phenotype result is derived from its supporting evidence.

4.1 Retrieving dated facts

A retrieval statement has this form:

```
variable => source.attribute.property.function().where(predicate);
```

The source is usually `eadv`. The attribute names the clinical fact, the property selects its value or date, and the function summarises the matching records. A filter is optional. For example:

```
egfr_latest => eadv.lab_bld_egfr.val.last();  
egfr_lowest => eadv.lab_bld_egfr.val.min();  
egfr_recent => eadv.lab_bld_egfr.val.min().where(dt > sysdate - 365);
```

These are three different questions about the same history. The last statement selects records from the preceding 365 days before taking the minimum. Within its filter, `dt` and `val` refer to the candidate record.

Dates often need to travel with their values. `lastdv()` produces a pair of outputs: a variable

named `egfr` becomes `egfr_val` and `egfr_dt`. This lets a later calculation check how old a result is. Other functions return the first observation, a maximum or minimum, a count, a mean, or a regression slope. The runtime also includes median, serialisation, and further date-value functions.

An attribute list can combine records from several codes. A trailing `%` matches an attribute prefix. These conveniences are useful for diagnosis families, but depend on the dictionary's coding conventions. An author should check the actual codes covered by a pattern before treating it as a clinical definition.

4.2 Applying a classification

Conditional statements use an ordered list of branches:

```
variable : {predicate => result}, {predicate => result}, {=> default};
```

The evaluator takes the first branch whose predicate holds. Order is therefore part of the rule. The following fragment calculates GFR and albuminuria categories from the latest available measurements. It expects eGFR in mL/min/1.73 m² and urine ACR in mg/mmol, using the category boundaries in KDIGO [\[Kidney Disease: Improving Global Outcomes \(KDIGO\) CKD Work Group, 2024\]](#).

```
egfr => eadv.lab_bld_egfr.val.last();
acr  => eadv.lab_ua_acr.val.last();

gfr_category : {egfr? or egfr < 0 => 'NA'},
               {egfr >= 90 => 'G1'},
               {egfr >= 60 => 'G2'},
               {egfr >= 45 => 'G3a'},
               {egfr >= 30 => 'G3b'},
               {egfr >= 15 => 'G4'},
               {=> 'G5'};

acr_category : {acr? or acr < 0 => 'NA'},
               {acr < 3 => 'A1'},
               {acr <= 30 => 'A2'},
               {=> 'A3'};
```

The complete ruleblock, including its metadata, is supplied in `examples/renal_categories.prb`. The first branch in each classification handles an absent or negative value. Without that branch, missing measurements could fall through to a category intended for an observed result. Here `NA` means that the example cannot assign a category.

This is a measurement classification, not a complete CKD diagnosis. Establishing CKD requires evidence of chronicity and the relevant clinical criteria; G1 or G2 alone does not establish the disease [\[Kidney Disease: Improving Global Outcomes \(KDIGO\) CKD Work Group, 2024\]](#). The example also leaves out treatment status and measurement age. Those decisions belong in a full clinical rule and its tests.

4.3 Expressions and missing values

Predicates support arithmetic, comparisons, **and**, **or**, **not**, set and range tests, and null checks. `x?` asks whether `x` is null; `x!?` asks whether it is present. Functions such as `round()`, `coalesce()`, `least()`, and `greatest()` support calculations over retrieved values. A result may be an expression rather than a literal. The dot in `{. => expression}` denotes an unconditional expression.

The JavaScript runtime aims to preserve SQL-like treatment of missing values. Arithmetic on a missing value should remain missing, and a comparison should not silently turn it into zero. Authors can choose a replacement with functions such as `coalesce()`, but that choice has clinical meaning. Substituting zero for an unknown laboratory result changes the question being answered.

A conditional with no matching branch and no default returns null. An explicit default usually makes the author's intention easier to inspect. For calculations using `sysdate`, reproducible testing also requires control of the evaluation date.

4.4 Composing definitions through shared constructs

A cardiovascular risk calculation may need the CKD classification already computed elsewhere. A binding imports it:

```
ckd_stage => rout_ckd.ckd_stage.val.bind();
```

The `rout_` prefix comes from the output tables produced by the original SQL compiler. In JavaScript, the binding refers to a result object from an earlier ruleblock. The dependency is visible in the source, allowing the execution order to place `ckd` before a rule that uses it.

This makes it possible to review a definition once and reuse its result in several places. It also means that changing an upstream definition can affect many outputs. A dependency graph helps identify those outputs; it does not decide whether the change is clinically appropriate.

The renal categories above illustrate two characteristics that could contribute to a broader renal phenotype. Other blocks could supply evidence about persistence, trajectory, or comorbidities. Together, their outputs could form a patient description used by both a registry and an individual-patient application. This is the intended pattern of composition; the category fragment alone does not supply all those characteristics.

Reuse also requires compatible meanings. A construct based on the latest available observation may not fit a definition requiring a measurement within a particular interval. A missing result may mean that evidence was not found, rather than that a condition is absent. Bindings make dependencies executable, but authors and reviewers must still establish that the definitions fit together.

4.5 Keeping documentation beside the definition

The block's description and activation metadata are supplied by `#define_ruleblock()`. Output names and descriptions use `#define_attribute()`. The `#doc()` directive attaches explanations and citation keys to a section of source.

These directives let the author record why a threshold was chosen while the definition is still in view. The execution parser skips the documentation directives; applications that display

the metadata need to extract it separately. A citation in a ruleblock also needs a maintained reference record if a reviewer is to follow it back to the evidence.

The language has no general loops or recursive procedures. More elaborate algorithms need external code or a suitable supported function. This keeps many routine calculations compact, at the cost of a deliberately limited range of expression.

5 Executing the same source in SQL and JavaScript

The line requesting the latest eGFR has a different workload in each setting. In a warehouse, it must find a result for every patient in the selected population. In an application, it may inspect a few dozen observations for one patient. The language describes the request; the backend chooses how to execute it.

Picorules uses one parser for these paths. It does not generate JavaScript source from the ruleblock. The JavaScript evaluator interprets the parsed rules directly.

5.1 Parsing and linking

The compiler first checks its inputs, including ruleblock identifiers and compilation options. It then normalises the source, removes comments, and splits statements at their terminators. Executable statements become one of three node types:

Node	Information retained	Example task
Fetch	Attribute list, property, aggregation, parameters, filter	Find the latest eGFR
Compute	Ordered predicates and result expressions	Assign a GFR category
Bind	Upstream block, variable, and property	Import the CKD classification

For a fetch, the parser preserves the attribute list as a list. For a compute statement, it preserves branch order. For a binding, it records which result must be available first. These details are needed by both execution paths.

Linking identifies variable references and dependencies between ruleblocks. A topological ordering puts prerequisites before their dependants, and a cycle is rejected. The SQL compiler uses this ordering to create tables in the required sequence. The JavaScript orchestrator derives its ordering from the bindings and passes completed result objects to later blocks.

The compiler can also select part of the rule library. A simple subset filters by name. Graph pruning can retain a requested output together with its prerequisites, or retain the rules affected by a selected input. These operations are useful when an application needs only a small part of a larger library. A name filter alone should not be mistaken for a complete dependency selection.

5.2 Generating SQL

For SQL execution, each statement becomes a query fragment. A latest-value retrieval typically uses `ROW_NUMBER()` to order observations within each patient. A count becomes a grouped

aggregation. A conditional becomes a `CASE` expression. A binding reads columns from an upstream output table.

The compiler then assembles the fragments into one result per patient. Left joins preserve patients for whom a particular observation is absent, so that the clinical rule can handle missingness explicitly.

The database dialect affects more than spelling:

Backend	Main assembly strategy	Output convention
Oracle	Common table expressions joined into a created table	<code>ROUT_<NAME></code>
PostgreSQL	Common table expressions with PostgreSQL expressions and casts	<code>ROUT_<NAME></code> in SQL text
SQL Server	Temporary tables for intermediate results, then a final materialised table	<code>SROUT_<name></code>

Date arithmetic, median calculations, string aggregation, and type conversions need dialect-specific treatment. Those operations are places where agreement has to be checked by executing the generated SQL. A successful compilation establishes that code was produced; it does not establish that each database returns the intended result.

Alongside the SQL, the compiler produces a manifest listing the execution order, dependencies, target tables, and output variables. A batch runner can use it to determine which statement creates which result. The generated queries remain available for inspection when a calculation behaves unexpectedly.

5.3 Evaluating in JavaScript

The JavaScript evaluator walks the parsed statements for one patient. A fetch calls `getRecords()`, applies its filter, and dispatches the selected aggregation. A compute statement evaluates its branches in order. A bind reads the required value from the supplied results of an upstream block.

`evaluateAll()` orders a set of blocks by their dependencies, evaluates them, and accumulates their results in a bindings map. A browser application can therefore use the same ruleblock relationships as a warehouse batch. It must first obtain the data those blocks need.

The evaluator has its own implementations of expression handling and aggregation. Sharing parsed source avoids maintaining separate clinical definitions, but does not eliminate differences between backend implementations. Null handling, date precision, tied timestamps, and numeric rounding are examples where tests need to establish the intended behaviour.

5.4 Choosing an execution path

SQL is the natural path when the clinical records already reside in an EADV warehouse and the task covers a population. The database can perform the joins and aggregations close to the data. JavaScript suits an application or service that already has one patient's records in memory. It is also convenient for tests built from small fixtures.

Development notes report roughly 16 ms to evaluate 153 active ruleblocks in memory for one test case, with higher elapsed times when retrieval is included. These are observations from the test harness, not a controlled benchmark. The workload, hardware, data volume, warm-up, and separation of parsing from evaluation need to be reported before the timings can support a performance comparison.

The useful property of the shared source is easier to demonstrate: an author changes the classification once, and both backends receive that change. Establishing that they interpret it identically requires a further step. The same fixtures should be run through JavaScript and each SQL dialect, with every expected output compared.

5.5 Software availability

The TypeScript reference implementation is openly available under the MIT licence as the npm package [picorules-compiler-js-core](#). Version 1.1.1 provides the compiler for SQL generation and the direct JavaScript evaluator. The FHIR R4 adapter is distributed separately as [picorules-adapter-fhir](#), version 0.2.0, also under the MIT licence. Both packages include TypeScript declarations and CommonJS and ES module builds.

The public [compiler repository](#) and [FHIR adapter repository](#) provide source code, tests, and development history. To install the identified releases for use with FHIR data:

```
npm install picorules-compiler-js-core@1.1.1 \
  picorules-adapter-fhir@0.2.0
```

Package versions identify the distributed software; source revisions identify the particular implementation examined or tested. Reproducing an evaluation also requires the corresponding rules, mappings, inputs, and evaluation date.

6 Connecting the rules to patient data

A rule asking for `lab_bld_egfr` assumes that someone has already decided what belongs in that series. In a warehouse, the decision may have been made during data preparation. With FHIR or openEHR, the adapter has to find the corresponding records. The rule can stay short because that work has a separate home.

6.1 EADV records

EADV represents a fact with four fields: entity, attribute, date, and value. For a patient, a series of eGFR results might therefore be several rows with the same attribute and different dates. The in-memory `EadvDataAdapter` filters such records by exact attribute names, lists, or prefix patterns.

This is useful for small test fixtures. A test can supply two measurements and check that `last()` selects the later one. It can then remove a measurement, change a date, or add a record to examine a boundary case. The fixture is small enough for a reviewer to understand without opening an EHR.

6.2 FHIR resources

The `FhirDataAdapter` accepts a FHIR R4 Bundle and indexes its resources. It maps Picorules attributes to terminology codes and resource types, then extracts the values and dates needed by the evaluator.

Some mappings are curated, such as those between laboratory attributes and LOINC codes. Others can be derived from attribute naming conventions for diagnoses or medications. A deployment can supply mapping overrides when the default vocabulary does not match its source.

Keeping these mappings outside the rule has a practical benefit. A change in the source’s coding can be handled in the mapping layer without editing every construct that uses the observation. It also creates a shared point of failure: an incorrect mapping can affect all the constructs and phenotype definitions that depend on it. Mapping tests belong beside rule tests.

The implementation can inspect parsed rules to identify requested attributes before fetching data. It then resolves available mappings into a set of FHIR searches. This can reduce retrieval, although the resulting search plan is only as complete as its mappings and the server’s supported search behaviour.

6.3 openEHR query results

The `OpenEhrDataAdapter` maps attributes to archetype paths and uses Archetype Query Language (AQL) to retrieve matching records. After retrieval, it presents the evaluator with the same value-and-date structure used by the other adapters.

This provides a useful test of the abstraction. The source structures differ substantially from FHIR, but the clinical rule can still request the latest eGFR through its attribute name. The mapping must fit the repository’s archetypes and templates; the small evaluator interface does not remove that requirement.

6.4 What the adapter tests establish

The development harness loads 153 active ruleblocks from a reference library of 168. Its fixtures include three synthetic patients and one de-identified TKC export represented as 3,498 FHIR resources. The openEHR comparison suite exercises the corresponding paths for these patients and checks selected outputs.

For the synthetic kidney disease fixture, the reported comparisons include:

Output	FHIR path	openEHR path
Latest eGFR	39	39
Latest ACR	45	45
Diabetes type	Type 2	Type 2
Charlson index	3	3

The test code also checks selected staging, medication, and cardiovascular outputs. For the larger TKC fixture it checks the number of evaluated blocks and selected CKD and diabetes results. The repository contains 25 end-to-end tests across the application and adapter scenarios, together with 74 FHIR adapter unit tests and 36 openEHR adapter unit tests.

These tests give concrete examples of source reuse across adapters. They compare selected results, rather than every variable from every block. Nor does comparing two JavaScript adapter paths establish agreement with generated SQL. A broader comparison should include all outputs, missing and unmapped data, and independently prepared representations of the same clinical facts.

7 Comparing the work an author has to do

The renal category example makes the trade-off visible. Its clinical branches are short because retrieval is expressed through a stable attribute name. The author can concentrate on the order of the thresholds. Someone still has to define what that attribute includes.

In raw SQL, retrieving the latest observation involves both the clinical selection and the mechanics of grouping records by patient. A typical fragment looks like this:

```
SELECT eid, val
FROM (
  SELECT eid, val,
    ROW_NUMBER() OVER (
      PARTITION BY eid ORDER BY dt DESC
    ) AS observation_order
  FROM eadv
  WHERE att = 'lab_bld_egfr'
) ranked
WHERE observation_order = 1;
```

This illustrates the operation that the compiler supplies. A production query also needs the surrounding population, tie handling, and agreed data preparation. The author of the Picorules statement benefits by leaving those database mechanics to the implementation.

In a CQL library using FHIR, retrieval may name an Observation value set and filter the resources before choosing a result. The extra detail can express useful distinctions, such as observation status and typed quantities. In Picorules, equivalent selection decisions may be made in the adapter or data preparation. Comparing the length of the visible rule alone would miss that work.

A practical comparison should therefore examine the whole definition, including its constituent constructs and data assumptions:

Question	What to inspect
Can a reviewer understand the rule?	Retrieval assumptions, branch order, missing data, and terminology
Can constructs be composed and reused?	Compatible time windows, missingness conventions, and meanings of shared outputs
Can the rule move to another source?	Mapping changes and tests needed to preserve meaning
Can it run in the required setting?	Database, application, runtime, and deployment constraints
Can a change be checked?	Fixtures, expected results, dependencies, and version history

Question	What to inspect
Can another team maintain it?	Documentation, tooling, skills, and governance

Picorules offers a compact way to express clinical constructs over dated attributes and compose them through shared outputs, with SQL and JavaScript execution paths. It asks the implementer to maintain the attribute vocabulary and its mappings. CQL offers a standard expression language and model descriptions; Arden and other formalisms bring their own structures for clinical knowledge. The right comparison is a set of equivalent tasks carried through to tested results.

No controlled authoring study or comparative benchmark is reported here. The examples explain the design choices and suggest what such a study should measure. They do not establish that one language is universally easier to read or cheaper to deploy.

8 Using AI to help write rules

Translating a guideline into a ruleblock takes more than copying its thresholds. The author must decide which observations qualify, how far back to look, and what an absent result means. Much of the effort lies in making those decisions explicit. Reusing an existing construct also requires checking that its meaning and assumptions fit the phenotype being defined.

A language model can help with an initial draft. Given a guideline passage and the local attribute dictionary, it can propose retrieval statements, organise branches, and suggest cases to test. The `picorules-agent` project explores this workflow, including compilation checks and composition with existing ruleblocks. Its effect on authoring time and error rates has not yet been measured in a controlled study.

The useful output is a piece of source that a reviewer can inspect. For the renal category example, a reviewer can point to the missing-value branch, check the ACR units, and ask how chronicity is handled. The discussion has an object that can be edited, versioned, and tested.

Passing the compiler is only one check. A syntactically valid rule can use the wrong attribute or encode an incomplete interpretation of a guideline. A review process needs to examine those choices and run fixtures with known expected results. Research on language models in clinical decision-making gives additional reason to evaluate such systems on the actual task rather than infer reliability from fluent output [Hager et al., 2024].

Once reviewed, the rule can be executed with a fixed implementation and specified inputs. Reproducibility requires the rule version, data, mappings, and evaluation date to be controlled. It does not establish that the rule is clinically correct: an incorrect threshold can be reproduced just as consistently as a correct one.

There is room for useful assistance here. An authoring tool could present a proposed rule alongside its source passage, list unresolved assumptions, and generate fixtures for clinician review. Picorules provides a compact artefact for that process. Assessing whether the process improves the work is a separate evaluation.

9 Territory Kidney Care: where the design began

Territory Kidney Care brings together information from services across Australia’s Northern Territory to support the identification and management of kidney disease. The setting includes remote communities, patients who receive care from several providers, and clinicians working with incomplete or dispersed records. The integration work is substantial in its own right [Gorham et al., 2024, Tan et al., 2025].

The published system description reports contributions from six public hospitals, more than 60 primary care services, and 11 Aboriginal Community-Controlled Health Services, drawing on 15 EHR platforms [Gorham et al., 2024]. Picorules ruleblocks run against the consolidated warehouse and supply derived results for clinical reports and population views.

9.1 Building with what was available

The original compiler was an Oracle PL/SQL package, `rman_pkg`. Ruleblock source was stored as text in a database table. The package parsed that text, assembled SQL fragments, and executed the result through dynamic SQL.

This arrangement suited the deployment restrictions in place when development began in 2019. It also kept the source, generated SQL, and output descriptions close together. An author could revise a rule and regenerate its queries within the database environment.

Several features survived the move to TypeScript: calculations over dated attributes, a query fragment for each step, and named outputs that later ruleblocks could reuse. The newer compiler adds SQL dialects and the JavaScript evaluator. The production history belongs to the original SQL deployment; the newer paths have their own, more limited evidence.

9.2 Published algorithm validation

Chen and colleagues evaluated TKC disease algorithms against blinded manual chart review in a stratified random sample of 288 patients drawn from a database of 48,569 individuals [Chen et al., 2022]. The reported results included:

Algorithm	Sensitivity	Specificity
CKD stage 3a or above	93%	97%
CKD stage 5	>99%	>99%
Diabetes	75%	97%
Hypertension	85%	88%
Cardiovascular disease	79%	96%

These results support the evaluated algorithms in that setting. They should not be read as validation of every current ruleblock or of a later compiler and adapter combination. Connecting a published result to a current deployment requires a record of the rule version, input preparation, and execution path.

9.3 Economic evaluation

A subsequent study modelled the cost-effectiveness of TKC decision support among First Nations Australians. It used observed patient data and projected outcomes over a 15-year horizon. The

scenario combining improved diagnosis and management produced an estimated incremental cost of \$3,427 per quality-adjusted life year gained [Chen et al., 2025].

This is evidence about a care intervention supported by the system. It is a modelled result under stated assumptions, and cannot be attributed to the programming language alone. The software operates within clinical services, data-sharing arrangements, and follow-up processes that determine whether an identified need leads to care.

9.4 Extending the original use

The warehouse remains the clearest example of Picorules in use. The same library also gives the newer execution paths a demanding test set: rules depend on other rules, records are missing, and the definitions span several clinical domains.

The SMART on FHIR harness and openEHR adapter work bring that library into additional settings. Their value to this paper is that they make the portability argument testable. The published clinical studies and the adapter comparisons answer different questions, and both are needed to understand the system’s progress.

10 Using phenotype definitions in clinical applications

Shared phenotype definitions can supply patient characteristics to several applications. Each application determines how those characteristics inform its own questions or actions. The execution path should fit the place where the result is needed. A nightly registry report and a consultation screen have different data access and timing requirements, even when they use the same clinical definition.

10.1 A population report

In the warehouse pattern, a scheduled job compiles the selected rule library, runs its SQL in dependency order, and exposes the resulting tables to reporting tools. Each output table associates a patient with the variables computed by that ruleblock.

The job needs a defined data snapshot and a record of the rules used. If a report changes after a guideline revision, those records help distinguish a change in the rule from a change in the underlying patient data. This is the setting from which Picorules developed.

10.2 A consultation application

A SMART on FHIR application obtains authorised patient data, constructs a Bundle, and passes it to the FHIR adapter. The JavaScript evaluator then returns results for display. The definitions can be evaluated in the browser once the necessary records have been fetched.

This avoids a separate server for rule evaluation. The application still needs to handle incomplete fetches, unavailable records, and the age of the data. If it retains records for offline use, the interface should make their date and completeness apparent. Fast evaluation cannot compensate for missing inputs.

10.3 A CDS Hooks service

A CDS Hooks service can perform the same evaluation after receiving context and available prefetch data. It can then construct response cards from selected outputs and their descriptions.

The service author has to decide which results warrant a card, how to explain them, and how to avoid repeated or unhelpful messages. Those are workflow decisions. The ruleblocks supply derived patient characteristics that the service can use, with their definitions available for inspection.

10.4 Another data source

For a source with neither an EADV view nor a FHIR interface, a team can implement the adapter contract and provide the required attribute mappings. The openEHR implementation is an example of this route.

A useful first step is a small fixture with a known result. The team can then compare the mapped records and outputs with an established path, adding date, unit, status, and missing-data cases as the mapping grows. This turns an integration claim into something that can be checked.

11 Where the design needs care

Picorules has a deliberately small view of patient data: named attributes with dated values. That view suits many laboratory summaries and classifications. It is less suitable when a phenotype definition depends on relationships that the attribute series does not preserve, such as a detailed chain of encounters, orders, and procedures. Flattening those relationships may lose information the rule needs.

The attribute dictionary is also part of the system's maintenance burden. Every new source must supply records with the expected meaning. An adapter can hide differences in storage structure, but differences in units, coding, status, and missingness still require decisions. Shared mappings reduce repeated work and make their correctness more consequential.

Composition has a similar boundary. A dependency graph records which outputs a rule uses, but does not establish that their time windows, missingness conventions, or clinical interpretations are compatible. A phenotype assembled from valid constructs can still be inappropriate for its intended use. Ruleblocks provide a mechanism for expressing those relationships; they do not constitute a complete model of clinical reasoning.

11.1 Evidence still needed

The examples illustrate mechanisms for composing definitions. A fuller evaluation should follow several phenotype families from their clinical requirements through their constituent constructs and resulting patient descriptions. Reusing a construct in different definitions, and revising a shared definition, would expose hidden assumptions and show where reuse requires adaptation.

The current test harness provides examples of agreement across JavaScript adapter paths. A fuller execution comparison should run the same fixtures through each SQL dialect and the JavaScript runtime, checking all outputs. Tied dates, nulls, rounding, and incomplete records deserve explicit cases. Agreement across backends would still leave the clinical definitions themselves to be validated.

Readability also needs evaluation. The examples are intended to be approachable to clinical informaticians, but familiarity with a short syntax is not the same as understanding its data assumptions. An authoring study should ask users to identify mistakes and make changes, as well as read finished examples.

11.2 Adoption and maintenance

Picorules is not an HL7 or ISO standard. A team adopting it takes responsibility for a smaller tool and governance ecosystem than those around established standards. The availability of the source helps inspection, but sustained maintenance, documentation, training, and release practices matter to a clinical service.

The JavaScript evaluator operates on one patient at a time. An application could orchestrate repeated evaluations, but the SQL path is the implementation designed for population processing in a warehouse. Neither path supplies a complete CDS platform: access control, workflow, monitoring, and clinical review belong to the deployment.

These boundaries help define where Picorules is useful. It is a practical candidate when a team wants to compose phenotype definitions from reusable constructs over dated clinical attributes and can maintain the definitions, mappings, and tests that give those constructs meaning.

12 The next useful steps

The next useful step is a set of worked investigations of phenotype composition. Each should state the intended clinical description, identify its constituent constructs, and trace their assumptions about evidence, time, and missingness. Reusing constructs across definitions and changing an upstream definition would help establish which relationships the language makes clear and which remain dependent on documentation or external review.

A reproducible comparison of the execution paths should accompany that work. It should publish the rule versions, fixtures, expected results, and commands needed to run JavaScript and generated SQL. Reporting disagreements would be as informative as reporting matches: each one identifies a part of the language whose behaviour needs clarification or correction.

Performance measurements can accompany that comparison. Parsing, data retrieval, and evaluation should be timed separately, with the hardware, patient history sizes, and repetition strategy recorded. Any comparison with another language should carry out the same clinical task and include its data preparation costs.

The language also needs a maintained formal grammar and a clearer account of its semantics. These would support editor tooling, better error messages, and independent implementations. For an author, a message that explains why a variable is unavailable may be more useful than another aggregation function.

Further adapter work should begin with a real integration need. An OMOP adapter, for example, would need both query support and a tested account of how its concepts map to the Picorules vocabulary. Extending coverage is useful when the additional mappings can be maintained and checked.

Finally, the authoring workflow deserves a study of its own. Clinicians and informaticians could work through equivalent tasks with and without AI assistance, measuring errors, review effort, and the quality of the resulting tests. That would provide evidence for the accessibility the language is intended to offer.

13 Conclusion

Picorules provides a language for expressing reusable clinical constructs and composing them into computable phenotype definitions. Its practical origins lie in maintaining clinical logic inside an Oracle database. Retrieving dated observations, deriving named characteristics, and reusing those characteristics across ruleblocks provide the mechanisms for building richer patient descriptions. The TypeScript implementation carries those mechanisms into several SQL dialects and a JavaScript evaluator.

The examples show how definitions can share named results and retain their source while the execution setting changes. They also show the responsibilities that remain. A short retrieval statement depends on a well-defined attribute. A meaningful composition depends on compatible assumptions, and a portable definition depends on mappings and execution semantics that preserve its intended behaviour.

TKC provides a production history and published evidence for the deployed algorithms. The newer adapter tests provide a starting point for evaluating reuse across data sources. Worked investigations of phenotype composition, a comprehensive comparison of backend results, and studies of authoring and review would make the case stronger.

My aim is to make clinical concepts explicit, reusable, and executable, so that richer descriptions of a patient can be assembled from definitions that remain open to review. A clinician should be able to follow a phenotype result through its constituent constructs to the supporting evidence. An implementer should be able to use those definitions in different settings while checking that their meaning is preserved. Picorules offers a working basis for those tasks, with the source and tests available to examine what it does.

Acknowledgements

The author acknowledges Dr Winnie Chen and the Territory Kidney Care team, whose published research informs the case study. Claude (Anthropic) and Codex (OpenAI) assisted with literature discovery, drafting, revision, and typesetting.

References

- Valentin Dinu et al. Guidelines for the effective use of entity-attribute-value modeling for biomedical databases. *International Journal of Medical Informatics*, 76(11–12):769–779, 2007.
- Jeffrey G. Klann et al. Data model harmonization for the All Of Us Research Program: Transforming i2b2 data into the OMOP common data model. *PLoS ONE*, 14(2):e0212463, 2019.
- B. Abell et al. Identifying barriers and facilitators to successful implementation of CDSS in hospitals: a NASSS framework-informed scoping review. *Implementation Science*, 18:32, 2023.
- E. Liberati et al. What hinders the uptake of computerized decision support systems in hospitals? *Implementation Science*, 12:113, 2017.
- Health Level Seven International. *Clinical Quality Language specification, version 1.5.3: Developer’s guide*, 2025. URL <https://cql.hl7.org/N1A/03-developersguide.html>.
- Pascal S Brandt, Richard C Kiefer, Jennifer A Pacheco, Prakash Adekkanattu, Evan T Sholle, Faraz S Ahmad, Jie Xu, Zhenxing Xu, Jessica S Ancker, Fei Wang, Yuan Luo, Guoqian Jiang, Jyotishman Pathak, and Luke V Rasmussen. Toward cross-platform electronic health record-driven phenotyping using Clinical Quality Language. *Learning Health Systems*, 4(4):Article e10233, 2020. doi: 10.1002/lrh2.10233.

- Pascal S. Brandt, Jennifer A. Pacheco, and Luke V. Rasmussen. Development of a repository of computable phenotype definitions using the Clinical Quality Language. *JAMIA Open*, 4(4):Article ooab094, 2021. doi: 10.1093/jamiaopen/ooab094.
- Peter Taber et al. New standards for clinical decision support: A survey of the state of implementation. *Yearbook of Medical Informatics*, 30(1):159–171, 2021.
- Andrey Soares, Robert A. Jenders, Robert Harrison, and Lisa M. Schilling. A comparison of Arden Syntax and Clinical Quality Language as knowledge representation formalisms for clinical decision support. *Applied Clinical Informatics*, 12(3):495–506, 2021. doi: 10.1055/s-0041-1731001.
- Natalia Iglesias et al. Comprehensive analysis of rule formalisms to represent clinical guidelines. *Artificial Intelligence in Medicine*, 103:101741, 2020.
- M. Peleg. Computer-interpretable clinical guidelines: A methodological review. *Journal of Biomedical Informatics*, 46(4):744–763, 2013.
- P. Papadopoulos et al. A systematic review of technologies and standards used in rule-based CDSS. *Health and Technology*, 12:713–729, 2022.
- Kidney Disease: Improving Global Outcomes (KDIGO) CKD Work Group. KDIGO 2024 clinical practice guideline for the evaluation and management of chronic kidney disease. *Kidney International*, 105(4S):S117–S314, 2024. doi: 10.1016/j.kint.2023.10.018. URL <https://kdigo.org/wp-content/uploads/2024/03/KDIGO-2024-CKD-Guideline.pdf>.
- P. Hager et al. Evaluation and mitigation of the limitations of large language models in clinical decision-making. *Nature Medicine*, 30:2613–2622, 2024.
- Gillian Gorham, Asanga Abeyaratne, Sam Heard, Liz Moore, Pratish George, Paul Kamler, Sandawana William Majoni, Winnie Chen, Bhavya Balasubramanya, Mohammad Radwanur Talukder, Sophie Pascoe, Adam Whitehead, Cherian Sajiv, Louise Maple-Brown, Nadarajah Kangaharan, and Alan Cass. Developing an integrated clinical decision support system for the early identification and management of kidney disease — building cross-sectoral partnerships. *BMC Medical Informatics and Decision Making*, 24:Article 69, 2024. doi: 10.1186/s12911-024-02471-w.
- Madeleine Sa Tan, Bhavini K Patel, Elizabeth E Roughead, Michael Ward, Stephanie E Reuter, Gregory Roberts, and Andre Q Andrade. Opportunities for clinical decision support targeting medication safety in remote primary care management of chronic kidney disease: A qualitative study in northern australia. *Journal of Telemedicine and Telecare*, 31(5):656–666, 2025. doi: 10.1177/1357633X231204545.
- Winnie Chen, Asanga Abeyaratne, Gillian Gorham, Pratish George, Vijay Karepalli, Dan Tran, Christopher Brock, and Alan Cass. Development and validation of algorithms to identify patients with chronic kidney disease and related chronic diseases across the Northern Territory, Australia. *BMC Nephrology*, 23:Article 320, 2022. doi: 10.1186/s12882-022-02947-9.
- Winnie Chen, Kirsten Howard, Gillian Gorham, Asanga Abeyaratne, Yuejen Zhao, Oyelola Adegboye, Nadarajah Kangaharan, Sean Taylor, Louise J. Maple-Brown, Samuel Heard, Mohammad Radwanur Talukder, Abdolvahab Baghbanian, Sandawana William Majoni, and Alan Cass. Cost-effectiveness of clinical decision support to improve CKD outcomes among first nations australians. *Kidney International Reports*, 10(2):549–564, 2025. doi: 10.1016/j.ekir.2024.10.028.